

유용한 메서드 모음 — 타입별 참조표

1. 언제 이 노트인가

다른 노트는 "이 상황엔 이 코드"로 상황별로 흩어져 있다. 이 노트는 반대로 **타입별로 모아놓은 사전**이다. "문자열에 이런 것도 되나?" 싶을 때 찾아본다. 외울 필요는 없고, 코딩 중 막히면 여기부터 훑는다. 각 항목은 **매개변수가 뭘 받는지, 결과가 정확히 어떤 모양으로 나오는지**까지 예시로 확인할 수 있게 적는다.

2. 문자열 (str) — 값을 바꿀 수 없다(불변), 메서드는 항상 "새 문자열"을 돌려준다

s.split(sep=None, maxsplit=-1)

- **sep**: 나눌 기준 문자(열). **생략하면** 공백 기준이고, 연속된 공백/개행도 하나로 취급하며 양 끝 공백은 무시한다.
- **maxsplit**: 최대 몇 번만 나눌지. 생략하면 끝까지 다 나눔.
- 결과: 항상 **리스트(list)**. 구분자 자체는 결과에 안 남는다.

```
"a,b,c".split(",")      # ['a', 'b', 'c']
"a,,b".split(",")      # ['a', '', 'b']    ← 구분자가 연달아 있으면 빈 문자열이 낄
" a b ".split()        # ['a', 'b']      ← sep 생략 시 연속 공백은 그냥 하나로 묶여 사라짐
"a,b,c".split(",", 1)  # ['a', 'b,c']    ← 처음 한 번만 나눔
```

sep.join(iterable)

- 호출 대상(**sep**)이 구분자 문자열, 인자로 **문자열들의 iterable**(리스트/튜플/제너레이터)을 받음.
- 리스트 안에 문자열이 아닌 게 있으면 **TypeError** — **join** 전에 **str()**이나 **map(str, ...)**로 변환 필요.
- 결과: 문자열 하나.

```
", ".join(['a', 'b', 'c'])      # "a,b,c"
"".join(['a', 'b', 'c'])       # "abc"      ← 구분자가 빈 문자열이면 그냥 이어붙임
", ".join(map(str, [1, 2, 3]))  # "1,2,3"    ← 숫자 리스트는 반드시 str로 변환
```

s.strip(chars=None) (lstrip, rstrip도 동일 규칙, 방향만 다름)

- **chars**: 제거할 **문자들의 집합**(문자열로 지정). 생략하면 공백류(스페이스·탭·개행) 제거.
- 주의: **chars**는 "이 순서로 된 문자열"이 아니라 "이 문자들 중 아무거나"를 양 끝에서 더 이상 없을 때까지 반복 제거하는 것. 중간 문자는 안 건드림.

```
" hi \n".strip()      # "hi"
"xxhixx".strip("x")   # "hi"
"xyxhixyx".strip("xy") # "hi"    ← 'x'든 'y'든 양 끝에서 계속 벗겨냄
```

```
"{{2}}".strip("{}")      # "2"    ← 양 끝의 { } 를 전부 벗김 (한 겹만 벗기려면 슬라이싱 s[1:-1] 사용)
```

s.replace(old, new, count=-1)

- old, new: 둘 다 문자열. count를 주면 앞에서부터 그 횟수만큼만 치환.
- 결과: 새 문자열. old가 없으면 원본과 동일한 문자열을 그대로 반환(에러 없음).

```
"aab".replace("a", "x")      # "xxb"
"aaab".replace("a", "x", 2)  # "xxab" ← 앞의 2개만 치환
```

s.find(sub, start=0, end=len(s)) vs s.index(sub, ...)

- 둘 다 부분문자열 sub가 처음 등장하는 인덱스를 반환. start/end로 탐색 범위 지정 가능.
- 차이: find는 못 찾으면 -1, index는 못 찾으면 ValueError 발생.

```
"hello".find("l")           # 2
"hello".find("z")           # -1    ← 에러 안 남, 조용히 -1
"hello".index("z")          # ValueError: substring not found
```

- 실수 포인트: if s.find(x): 로 쓰면 안 됨 — 찾은 위치가 0이면 if 0:은 False라 "못 찾음"처럼 오작동. 반드시 if s.find(x) != -1:.

s.count(sub)

- sub가 겹치지 않게 몇 번 나오는지.

```
"aaba".count("a")           # 3
"aaaa".count("aa")          # 2    ← "aa"+"aa"로 겹치지 않게 센 것. "aaa"였다면 1
```

대소문자/판별계 — upper(), lower(), isdigit(), isalpha(), isupper(), islower()

- 인자 없이 호출, 전부 원본은 안 바꾸고 새 값(문자열 또는 True/False)을 반환.

```
"AbC".upper()               # "ABC"
"AbC".lower()               # "abc"
"a1b2".isdigit()            # False ← 문자열 "전체"가 숫자로만 이루어져야 True
"123".isdigit()             # True
"".isdigit()                 # False ← 빈 문자열은 항상 False
```

s.startswith(prefix) / s.endswith(suffix)

- prefix/suffix 자리에 튜플을 넣으면 "여러 개 중 하나라도 맞으면" 확인 가능.

```
"report.pdf".endswith(".pdf")          # True
"report.pdf".endswith((".pdf", ".doc")) # True ← 튜플로 여러 개 동시 검사
```

s.zfill(width)

- **width**: 만들고 싶은 전체 길이. 원본이 더 길면 그대로 둠(잘리지 않음).
- 앞에 0을 채우되, 맨 앞이 +/- 부호면 부호는 그대로 맨 앞에 두고 그 뒤부터 0 채움.

```
"5".zfill(3)      # "005"
"-5".zfill(3)     # "-05"
"12345".zfill(3)  # "12345" ← 이미 더 길면 그대로
```

ord(c) / chr(n)

- **ord**: 문자 1개 → 유니코드(아스키) 코드값(정수). 2글자 이상 넣으면 **TypeError**.
- **chr**: 정수 → 그 코드값에 해당하는 문자 1개.

```
ord('a')      # 97
ord('A')      # 65
chr(97)       # 'a'
chr(ord('a') + 1) # 'b' ← 알파벳 밀기(시저 암호 등)에 자주 쓰는 조합
```

3. 리스트 (list) — 값을 바꿀 수 있다(가변)

arr.append(x)

- **x** 자리에 **뭐든 하나** (리스트 자체를 넣어도 통째로 원소 하나로 들어감).
- 반환값은 **None** — **arr = arr.append(x)**로 쓰면 **arr**가 **None**이 되어버리는 흔한 실수.

```
a = [1, 2]
a.append(3)      # a는 [1, 2, 3]으로 바뀜 (append 자체의 반환값은 None)
a.append([4, 5]) # a는 [1, 2, 3, [4, 5]] ← 리스트가 통째로 원소 하나
```

arr.extend(iterable)

- 인자로 받은 **iterable**의 **원소들을 하나씩 풀어서** 뒤에 붙임.

```
a = [1, 2]
a.extend([3, 4]) # a는 [1, 2, 3, 4]
a.extend("ab")   # 문자열도 iterable이라 문자 단위로 풀림 → [..., 'a', 'b']
```

arr.insert(i, x)

- **i**: 삽입할 위치(인덱스). **i**가 범위를 넘어가도 에러 없이 맨 앞/뒤에 들어감.

```
a = [1, 2, 3]
a.insert(1, 99)    # a는 [1, 99, 2, 3]
a.insert(100, 5)   # 범위 초과해도 에러 안 나고 맨 끝에 추가 → [..., 5]
```

arr.remove(x)

- 값이 **x**와 같은 첫 번째 원소를 찾아서 삭제. 인덱스가 아니라 값으로 찾는다.
- 없으면 **ValueError**.

```
a = [3, 1, 2, 1]
a.remove(1)    # 첫 번째 1만 지워짐 → [3, 2, 1]
```

arr.pop(i=-1)

- **i** 생략 시 맨 뒤 원소. 지정하면 그 인덱스의 원소.
- 삭제와 동시에 그 값을 반환 (스택/큐 구현에 자주 씬).

```
a = [1, 2, 3]
x = a.pop()    # x=3, a=[1,2]
y = a.pop(0)   # y=1, a=[2] ← 인덱스 0 pop은 O(N), 큐로 쓰면 안 됨(deque 쓸 것)
```

arr.index(x) / arr.count(x)

- **index**: 값 **x**가 처음 나오는 인덱스, 없으면 **ValueError**.
- **count**: 값 **x**가 몇 번 나오는지(개수, 정수).

```
[10, 20, 20, 30].index(20)    # 1
[10, 20, 20, 30].count(20)    # 2
```

arr.reverse() vs arr[::-1]

- **reverse()**: 원본 자체를 뒤집음, 반환값은 **None**.
- **arr[::-1]**: 슬라이싱으로 뒤집힌 새 리스트를 만들, 원본은 그대로.

```
a = [1, 2, 3]
a.reverse()      # a는 [3, 2, 1]로 바뀜, 반환값 None
b = [1, 2, 3]
c = b[::-1]      # c는 [3, 2, 1], b는 그대로 [1, 2, 3]
```

arr.copy()

- 얕은 복사(shallow copy) — 리스트 자체는 새로 만들어지지만, 안에 든 리스트/딕셔너리 같은 가변 객체는 참조를 공유함.

```
a = [[1, 2], 3]
b = a.copy()
b[1] = 99        # a는 안 바뀜 ([1,2], 3])
b[0].append(9)   # a도 바뀜! a는 [[1,2,9], 3] ← 안쪽 리스트는 같은 객체를 공유
```

sum(arr, start=0) / max(arr, key=..., default=...) / min(arr, ...)

- sum**: 두 번째 인자로 시작값 지정 가능(기본 0).
- max/min**: **key=**로 비교 기준 함수 지정, **default=**로 빈 iterable일 때 에러 대신 반환할 값 지정.

```
sum([1, 2, 3])          # 6
sum([1, 2, 3], 100)     # 106
max([], default=0)      # 0 ← default 없으면 빈 리스트에 max() 호출 시
                        ValueError
max(["a", "bb", "c"], key=len) # "bb" ← 길이 기준 최댓값
```

x in arr

- 리스트를 앞에서부터 순회하며 비교 → **O(N)**. 존재 확인을 자주/크게 할 거면 **set**으로 바뀌서 **in** 쓰면 **O(1)**.

```
3 in [1, 2, 3]          # True (내부적으로 리스트 끝까지 훑을 수 있음)
```

append/extend 차이 정리: `[1,2].append([3,4]) → [1, 2, [3, 4]]`(통째로 하나 추가),
`[1,2].extend([3,4]) → [1, 2, 3, 4]`(풀어서 추가).

4. 딕셔너리 (dict)

d.get(key, default=None)

- 키가 있으면 그 값, **없어도 에러 없이 default**(생략 시 **None**) 반환.

```
d = {"a": 1}
d.get("a")      # 1
```

```
d.get("b")      # None
d.get("b", 0)    # 0
```

d[key]

- 키가 없으면 `KeyError`. 있을지 확신 없을 땐 `.get()`을 쓰는 게 안전.

d.keys() / d.values() / d.items()

- 각각 키들 / 값들 / (키, 값) 튜플들을 순회 가능한 **뷰(view) 객체**로 반환 (리스트는 아니지만 `for`로 순회 가능하고 `list()`로 감싸면 리스트가 됨).

```
d = {"a": 1, "b": 2}
list(d.keys())    # ['a', 'b']
list(d.values())  # [1, 2]
list(d.items())   # [('a', 1), ('b', 2)]
for k, v in d.items(): # 키·값 동시에 필요할 때 표준 패턴
    print(k, v)
```

key in d

- 키 기준으로 존재 확인 (값 기준 아님). $O(1)$.

```
"a" in {"a": 1}    # True (키 'a' 존재)
1 in {"a": 1}     # False (값 1은 안 봄, 키만 봄)
```

d.pop(key, default=...)

- 키를 삭제하면서 그 값을 반환. `default`를 주면 키가 없을 때 에러 대신 그 값 반환 (안 주면 없을 때 `KeyError`).

```
d = {"a": 1}
d.pop("a")      # 1, d는 {} 가 됨
d.pop("z", -1)  # -1, 에러 안 남
```

d.setdefault(key, default)

- 키가 있으면 그 값을 그대로 반환. 없으면 `default`로 새로 설정하고 그 값을 반환.
- "없으면 초기화하고 시작"하는 로직을 한 줄로 (그룹핑에 자주 씬).

```
d = {}
d.setdefault("a", []).append(1) # d는 {"a": [1]}
d.setdefault("a", []).append(2) # d는 {"a": [1, 2]} ← 이미 있으니 default 무시하
고 그 리스트에 append
```

d.update(other)

- **other**(딕셔너리 또는 (key, value) 쌍들의 iterable)의 내용을 **병합**. 같은 키는 덮어씀.

```
d = {"a": 1, "b": 2}
d.update({"b": 20, "c": 3}) # d는 {"a": 1, "b": 20, "c": 3}
```

5. 집합 (set) — 순서 없음, 중복 없음

s.add(x) / s.remove(x) / s.discard(x)

- **add**: 원소 하나 추가(이미 있으면 아무 일도 안 일어남).
- **remove**: 삭제, **없으면 KeyError**.
- **discard**: 삭제, 없어도 에러 안 남(조용히 무시).

```
s = {1, 2}
s.add(2) # 이미 있어서 그대로 {1, 2}
s.remove(2) # {1}
s.remove(99) # KeyError!
s.discard(99) # 에러 없음
```

집합 연산자: &(교집합), |(합집합), -(차집합), ^(대칭차집합)

- 전부 두 **set끼리** 연산, 결과도 set.

```
a, b = {1, 2, 3}, {2, 3, 4}
a & b # {2, 3} ← 둘 다에 있는 것
a | b # {1, 2, 3, 4} ← 합침(중복 제거)
a - b # {1} ← a에만 있는 것
b - a # {4} ← 방향 주의! a-b와 b-a는 다름
a ^ b # {1, 4} ← 한쪽에만 있는 것들 (교집합 제외)
```

a <= b (부분집합 확인, issubset과 동일)

```
{1, 2} <= {1, 2, 3} # True
```

집합 리터럴 함정

```
{(1, 2)} # 튜플 (1,2) 하나를 원소로 갖는 집합 - 올바른
set((1, 2)) # {1, 2} - 튜플을 "풀어서" 두 원소로! (의도와 다를 수 있음)
set([(1, 2)]) # {(1, 2)} - 리스트로 감싸면 튜플이 원소 하나로 유지됨
```

6. 자주 쓰는 내장 함수 (타입 무관)

sorted(iterable, key=None, reverse=False)

- 원본은 안 건드리고 **정렬된 새 리스트**를 반환 (리스트의 `.sort()`는 반대로 원본 자체를 바꾸고 반환값 `None`).
- `key`: 각 원소를 이 함수에 통과시킨 결과를 기준으로 비교.

```
sorted([3, 1, 2])                # [1, 2, 3]
sorted([3, 1, 2], reverse=True)  # [3, 2, 1]
sorted(["bb", "a", "ccc"], key=len) # ['a', 'bb', 'ccc']
sorted([(1, 'b'), (1, 'a')], key=lambda x: (x[0], x[1])) # [(1, 'a'), (1, 'b')] ← 다
중 기준(튜플)
```

all(iterable) / any(iterable)

- `all`: 원소가 **전부** 참이어야 `True` (빈 iterable이면 `True` — "거짓인 게 하나도 없다"라서).
- `any`: 원소 중 **하나라도** 참이면 `True` (빈 iterable이면 `False`).

```
all([1, 2, 3])    # True  (0이 아닌 수는 다 참)
all([1, 0, 3])    # False (0이 하나 있음)
all([])           # True
any([0, 0, 1])    # True
any([])           # False
```

reversed(iterable)

- 뒤집힌 **순회 객체(iterator)**를 반환 — 리스트가 아니라서 그대로 인덱싱은 안 됨, `list()`로 감싸거나 `for`로 바로 순회.

```
list(reversed([1, 2, 3])) # [3, 2, 1]
for x in reversed("abc"): # 'c', 'b', 'a' 순서로 순회 (문자열도 가능)
    print(x)
```

divmod(a, b)

- `(a // b, a % b)`를 튜플 하나로 동시에.

```
divmod(17, 5)    # (3, 2) ← 17//5=3, 17%5=2
q, r = divmod(17, 5) # q=3, r=2
```

enumerate(iterable, start=0)

- (인덱스, 값) 쌍을 순서대로 만들어내는 iterator. `start`로 시작 인덱스 지정 가능(기본 0).

```
for i, x in enumerate(["a", "b"]):      # (0, 'a'), (1, 'b')
    print(i, x)
list(enumerate(["a", "b"], start=1))    # [(1, 'a'), (2, 'b')]
```

zip(*iterables)

- 여러 iterable을 같은 인덱스끼리 묶어서 튜플로. 가장 짧은 것 기준으로 멈춤(에러 없음).

```
list(zip([1,2,3], ['a','b']))           # [(1, 'a'), (2, 'b')] ← 3번째 1은 버려짐
list(zip([1,2], [3,4], [5,6]))          # [(1,3,5), (2,4,6)]
```

7. 자주 쓰는 모듈 — 코테 필수 4종 + 기타

collections.Counter(iterable)

- 인자로 iterable(리스트, 문자열, 제너레이터 등)을 받아 원소별 등장 횟수를 dict처럼 담은 객체를 만든다. 존재하지 않는 키를 조회해도 **KeyError** 없이 0을 반환한다(일반 dict와 다른 점).

```
from collections import Counter
c = Counter("aab")           # Counter({'a': 2, 'b': 1})
c["a"]                       # 2
c["z"]                       # 0 ← 없는 키인데 에러 안 남 (일반 dict였으면 KeyError)
c.values()                   # dict_values([2, 1]) ← 개수들만
```

- Counter(a) - Counter(b)**: a에만 있거나 a가 더 많은 만큼만 남고, 음수 결과는 버려짐 (0 이하는 아예 안 보임).

```
Counter([1,1,2]) - Counter([1])      # Counter({1: 1, 2: 1})
Counter([1]) - Counter([1,1,2])      # Counter() ← 음수가 되니 전부 사라짐, 빈
Counter
```

- .most_common(n)**: 개수가 많은 순으로 정렬된 (값, 개수) 튜플의 리스트. n 생략 시 전부.

```
Counter("aabbcc").most_common(2)     # [('b', 3), ('a', 2)]
```

collections.deque(iterable=[])

- 양쪽 끝에서 추가/삭제가 전부 O(1)인 큐. **list.pop(0)**은 O(N)이라 큐로 쓰면 안 되고 이걸 써야 함.

```
from collections import deque
q = deque([1, 2, 3])
```

```
q.append(4)          # 오른쪽에 추가 → deque([1,2,3,4])
q.appendleft(0)      # 왼쪽에 추가 → deque([0,1,2,3,4])
q.pop()             # 오른쪽에서 꺼냄, 반환 4 → deque([0,1,2,3])
q.popleft()         # 왼쪽에서 꺼냄, 반환 0 → deque([1,2,3])
```

`collections.defaultdict(default_factory)`

- 인자로 **타입(또는 인자 없이 호출 가능한 함수)**을 받는다 (`list`, `int`, `set` 등).
- 없는 키에 접근하면 `KeyError` 대신 `default_factory()`의 결과로 **자동 생성**해서 넣어줌.

```
from collections import defaultdict
d = defaultdict(list)
d["a"].append(1)    # "a" 키가 없었지만 자동으로 [] 생성 후 append → {'a': [1]}
d["a"].append(2)    # {'a': [1, 2]}

e = defaultdict(int)
e["x"] += 1         # 없던 키가 0에서 시작해서 1 → {'x': 1}
```

`itertools.accumulate(iterable, func=operator.add)`

- `func` 생략 시 누적 **합**. 다른 함수(`max` 등)를 주면 누적 그걸로.
- 결과는 **제너레이터**라서 `list()`로 감싸야 실제 리스트가 됨.

```
from itertools import accumulate
list(accumulate([1, 2, 3, 4]))    # [1, 3, 6, 10]    ← 누적합
list(accumulate([1, 3, 2, 5], max)) # [1, 3, 3, 5]    ← 누적 최대값
```

`itertools.permutations(iterable, r=None)`

- `iterable`의 원소들로 만들 수 있는 **순열**(순서가 다르면 다른 경우로 참). `r` 생략 시 전체 길이만큼.
- 결과는 **튜플들을 만들어내는 제너레이터**.

```
from itertools import permutations
list(permutations([1, 2, 3], 2))
# [(1,2), (1,3), (2,1), (2,3), (3,1), (3,2)] ← 총 3*2=6개
```

`itertools.combinations(iterable, r)`

- `r`은 **필수**(몇 개씩 뽑을지). 순서 무관, 같은 조합 중복 없음.

```
from itertools import combinations
list(combinations([1, 2, 3], 2))
# [(1,2), (1,3), (2,3)] ← (2,1) 같은 건 안 나옴 (순서 무관이라 하나로 참)
```

itertools.combinations_with_replacement(iterable, r)

- combinations와 같은데 자기 자신과의 중복도 허용.

```
from itertools import combinations_with_replacement as cwr
list(cwr([1, 2], 2))    # [(1,1), (1,2), (2,2)]
```

itertools.product(*iterables, repeat=1)

- 여러 iterable의 데카르트 곱 — 중첩 for문을 한 줄로.

```
from itertools import product
list(product([1, 2], ['a', 'b']))
# [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
list(product([0, 1], repeat=3))    # 길이 3짜리 0/1 조합 8개 — 완전탐색에 유용
```

itertools.chain(*iterables)

- 여러 iterable을 하나로 이어서 순회(새 리스트를 만드는 게 아니라 이어서 순회만).

```
from itertools import chain
list(chain([1, 2], [3, 4], "ab"))    # [1, 2, 3, 4, 'a', 'b']
```

itertools.groupby(iterable, key=None)

- 연속으로 같은 값끼리 묶는다 — 정렬이 안 되어 있으면 떨어진 같은 값끼리는 안 묶임 (미리 정렬 필요할 때가 많음).

```
from itertools import groupby
[(k, list(g)) for k, g in groupby("aaabbc")]
# [('a', ['a', 'a', 'a', 'a']), ('b', ['b', 'b']), ('c', ['c'])]
[(k, list(g)) for k, g in groupby("aabaa")]
# [('a', ['a', 'a', 'a']), ('b', ['b']), ('a', ['a', 'a', 'a'])] ← 정렬 안 하면 'a'가 두 그룹으로 쪼개짐
```

functools.reduce(function, iterable, initializer=...)

- function은 인자 2개를 받아 하나를 반환하는 함수. 왼쪽부터 차례로 누적 적용.
- initializer 생략 시 iterable의 첫 원소를 시작값으로 씀.

```
from functools import reduce
from operator import mul
reduce(lambda a, b: a + b, [1, 2, 3, 4])    # 10 ← ((1+2)+3)+4
```

```
reduce(mul, [1, 2, 3, 4])          # 24 ← ((1*2)*3)*4
reduce(lambda a, b: a + b, [1, 2, 3], 100)  # 106 ← 시작값 100부터
```

functools.lru_cache(maxsize=128)

- 함수 위에 데코레이터로 붙여서 **같은 인자로 다시 호출하면 계산 없이 캐시된 값을 즉시 반환**.
- 인자는 **불변 타입**(정수, 문자열, 튜플 등)이어야 함 — 리스트를 인자로 주면 **TypeError**.

```
from functools import lru_cache

@lru_cache(maxsize=None)  # None이면 무제한 캐시
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)

fib(30)  # 캐시 없으면 지수시간, 있으면 순식간
```

functools.cmp_to_key(cmp_function)

- "두 원소를 비교해서 -1/0/1을 반환하는 함수"를 **sorted**의 **key=**가 받을 수 있는 형태로 변환.
- cmp_function(a, b)**는 a가 앞에 와야 하면 음수, 뒤에 와야 하면 양수, 같으면 0.

```
from functools import cmp_to_key

def compare(a, b):
    # 이어붙였을 때 더 큰 쪽이 앞에 오도록 (가장 큰 수 만들기류)
    if a + b > b + a:
        return -1
    elif a + b < b + a:
        return 1
    return 0

sorted(["3", "30", "34"], key=cmp_to_key(compare))
# ['34', '3', '30'] ← "34"+"3"+"30" 이어붙이면 가장 큰 수
```

operator.mul / add / sub (그 외 다수)

- 람다 대신 쓰는 **함수형 연산자** — **reduce**나 **key=**에 바로 넘길 때 편함.

```
from operator import mul, add
reduce(mul, [1, 2, 3])  # 6, reduce(lambda a,b: a*b, ...)과 동일
```

operator.itemgetter(*items)

- 인덱스(또는 딕셔너리 키)를 받아서, 그 위치의 값을 꺼내는 **함수를 만들어 반환**.
- `sorted`의 `key=`에 람다 대신 자주 씀 (속도도 약간 더 빠름).

```
from operator import itemgetter
data = [(1, 'b'), (2, 'a')]
sorted(data, key=itemgetter(1))      # [(2, 'a'), (1, 'b')] ← 두 번째 값 기준 정렬
sorted(data, key=lambda x: x[1])    # 위와 완전히 동일한 결과
itemgetter(0, 1)((10, 20, 30))     # (10, 20) ← 여러 인덱스 동시에 꺼내면 튜플로
```

`operator.attrgetter(*attrs)`

- 객체의 **속성 이름**을 받아 그 속성값을 꺼내는 함수 생성. 코테에서 커스텀 클래스를 거의 안 쓰기 때문에 드물게 쓰임 — `itemgetter`가 훨씬 자주 나옴.

기타 — 각자 전용 노트에서 더 자세히

- `heapq.heappush(heap, x)` / `heapq.heappop(heap)` — 리스트를 최소힙처럼 사용. 최댓값 힙은 `-x`로 부호를 반전해 넣었다 꺼낼 때 다시 반전. → [[15-힙]]
- `bisect.bisect_left/right(정렬된_배열, x)` — 정렬된 배열에서 값이 들어갈 위치를 $O(\log N)$ 에 탐색. → [[19-이분탐색]]
- `math.gcd(a, b)`, `math.sqrt(x)`, `math.inf`(무한대, 최솟값 초기화 등에 사용), `math.ceil/math.floor` — 올림/내림.
- `re.findall(pattern, s)` — 정규식으로 문자열에서 패턴에 맞는 부분을 전부 리스트로 추출. → [[12-문자열파싱]]

8. 자주 하는 실수

- `.remove()`는 **값**으로 찾아 지우고, `.pop(i)`는 **인덱스**로 지운다 — 헛갈리면 엉뚱한 원소가 삭제됨
- `d[k]`로 없는 키에 접근해 `KeyError` — 있을지 확신 없으면 `.get()` 사용
- `s.find()`는 없으면 -1을 돌려줄 뿐 에러가 안 나서, 이걸 `if s.find(x):`처럼 참/거짓 판단에 잘못 쓰면 버그 (0번 인덱스에서 찾아도 `if 0:`은 `False`로 취급됨) — 반드시 `if s.find(x) != -1:`처럼 명시 비교
- `arr.append(x)`, `arr.sort()`, `arr.reverse()` 등은 반환값이 `None` — `arr = arr.sort()`처럼 대입하면 `arr`가 `None`이 되어버림. 원본을 바꾸는 메서드는 **대입하지 말고 그냥 호출만**
- `list.copy()`는 얇은 복사 — 리스트 안에 리스트가 있으면 안쪽은 여전히 공유됨